Harbor Watch – NVDEC + Socket.IO Integration Explained

A first-time walkthrough of the integrated code, covering architecture, NVDEC, Socket.IO, new terminology and issues to fix before running.

1. Overall Understanding of the Project

The Harbor Watch application has four main responsibilities:

- Receive video streams from CCD and IR cameras.
- Decode video using NVIDIA GPU hardware.
- Perform detection, classification and analytics.
- Send processed video frames and detection information to the frontend using Socket.IO.

```
CAMERA INPUT
     |
+----+-----+
|          |
CCD Cams   IR Cams
|          |
RTSP       RTSP
+----+-----+
     |
NVDEC Decoder -> GPU Video Frames -> Frame Retrieval
     |
Image Preprocessing -> Split and Concatenate
     |
Detection Model -> Classification Model
     |
ROI / Analytics -> Result Generation
     |
JPEG Encoding -> Base64 Encoding
     |
Socket.IO
 +---------+---------+
 |                   |
/front              /rear
 |                   |
FRONT UI            REAR UI
```

Important: NVDEC handles video decoding. Socket.IO handles communication between the Python backend and the frontend. They are two separate components.

2. Understanding NVDEC Integration

2.1 What is NVDEC?

NVDEC stands for NVIDIA Video Decoder. It is a dedicated hardware video decoder available on supported NVIDIA GPUs.

With CPU decoding:

```
RTSP Video -> CPU Decoding -> CPU Frames -> GPU Transfer -> GPU Processing
```

With NVDEC:

```
RTSP Video -> NVIDIA NVDEC -> GPU Frames -> GPU Processing
```

The main advantage is reducing CPU decoding workload and avoiding unnecessary memory transfers. NVDEC does not perform object detection or classification. It only decodes compressed video into image frames.

2.2 New Imports

```
import av
import PyNvVideoCodec as nvc
```

Term	Meaning	Purpose
av	PyAV library	Reads video packets and stream information
PyNvVideoCodec	NVIDIA video decoding library	Provides access to NVDEC
nvc	Import alias	Short name for PyNvVideoCodec

What is PyAV? A Python library built around FFmpeg. It can open video streams, read video packets, extract codec information and demultiplex video streams. In the NVDEC implementation, PyAV obtains compressed packets which are passed to NVIDIA's decoder.

What is demuxing? Separating audio, video and other data from a multimedia container or stream.

```
Compressed Video Stream -> Demuxer -> Video Packets -> NVDEC -> Decoded
Frames
```

This is needed because the decoder expects compressed video packets, not the original RTSP stream as an ordinary image.

2.3 PacketData

The integrated code introduces:

```
_PacketData = nvc._PyNvVideoCodec.PacketData
```

This creates a shorter reference to NVIDIA's packet data structure. The next function is:

```
def make_packet(buf):
    arr = np.frombuffer(buf, np.uint8)
    packet = _PacketData()
    packet.bsl_data = arr.ctypes.data
    packet.bsl = arr.nbytes
    return packet, arr
```

Line	What it does
`np.frombuffer(buf, np.uint8)`	Converts the byte buffer into a NumPy array without initially copying the underlying data
`_PacketData()`	Creates an NVIDIA packet structure
`packet.bsl_data = arr.ctypes.data`	Provides the memory address of the compressed packet data
`packet.bsl = arr.nbytes`	Specifies the packet's size in bytes

Why return both packet and arr? The NumPy array must remain alive while the decoder accesses its memory. If Python releases that memory too early, the decoder could access invalid memory. This is called a buffer lifetime or memory ownership issue.

2.4 CUDA Synchronization

```
def sync_current_stream():
    event = torch.cuda.Event()
    event.record()
    event.synchronize()
```

- `torch.cuda.Event()` creates a CUDA event.

- `record()` records the event on the current CUDA stream.
- `synchronize()` waits until the recorded GPU work has completed.

GPU operations are often asynchronous, so Python can continue executing before the GPU finishes an operation. Synchronization ensures certain operations have completed before the program proceeds.

Note that this helper is not a substitute for synchronizing the particular CUDA stream that produced a frame. The actual synchronization requirements depend on the decoder and tensor ownership.

3. RTSPReader – The Most Important Component

In the original hw_v3.py there was an RTSPReader class. In nvdec (2).py there is an NVDEC-based RTSPReader. Its responsibility is to:

1. Connect to the camera.
2. Receive compressed video packets.
3. Decode packets using NVDEC.
4. Store the latest decoded frame.
5. Allow the inference thread to retrieve that frame.

```
RTSPReader
  +-- Open Camera
  +-- Read Packets
  +-- NVDEC Decode
  +-- Convert to GPU Tensor
  +-- Store Latest Frame
  +-- Provide Frame to Inference
```

Important functions

Function	What it does
`__init__()`	Initializes the reader
`_open()`	Opens the video stream and decoder
`_run()`	Runs the packet decoding loop
`_reader_loop()`	Manages decoding and reconnection
`get_latest()`	Returns the latest GPU frame
`get_latest_bgr()`	Converts the GPU frame into CPU NumPy BGR

Function	What it does
`stop()`	Stops the reader thread

What is a thread?

A thread is an independent execution path within a program. Four cameras can have four independent decoding threads.

```
        MAIN PROGRAM
             |
   +---------+---------+
   |         |         |
 Thread 1  Thread 2  Thread 3
 Front CCD Rear CCD  Front IR
```

This allows video decoding to continue while inference is happening.

What is a latest-frame buffer?

Suppose the camera generates 30 frames per second but the detection model processes only 15. If every frame is queued, the queue keeps growing and the UI eventually shows old frames. A latest-frame buffer stores only the newest frame; older unprocessed frames are replaced.

```
Frame 1, 2, 3, 4, 5  ->  Latest Frame = Frame 5
```

This is particularly useful for real-time surveillance.

What is torch.from_dlpack()?

```
frame = torch.from_dlpack(surf).clone()
```

DLPack is a standard for sharing tensor memory between different libraries. It allows PyTorch to access the decoded GPU surface without first converting the frame into a CPU NumPy array.

`clone()` creates a separate copy of the tensor, because the decoder may reuse its original memory for subsequent frames. The clone protects the frame being processed.

4. What Changed in the GPU Inference Thread?

```
def gpu_inference_thread(model1, sio, cam_CCD_FRONT, cam_CCD_REAR,
cam_IR_FRONT, cam_IR_REAR):
```

This function coordinates the entire processing pipeline.

Variable	Camera
cam_CCD_FRONT	Front CCD
cam_CCD_REAR	Rear CCD
cam_IR_FRONT	Front IR
cam_IR_REAR	Rear IR

4.1 Camera mode selection

```
if sh.CAMERA_MODE == 1:
    frame1 = cam_CCD_FRONT.get_latest_bgr()
    frame2 = cam_CCD_REAR.get_latest_bgr()
elif sh.CAMERA_MODE == 0:
    frame1 = cam_IR_FRONT.get_latest_bgr()
    frame2 = cam_IR_REAR.get_latest_bgr()
```

- `CAMERA_MODE = 1` selects CCD.
- `CAMERA_MODE = 0` selects IR.

`get_latest_bgr()` performs `return frame.flip(-1).cpu().numpy()`, which:

1. Flips RGB channels into BGR order.
2. Transfers the frame from GPU memory to CPU memory.
3. Converts it into a NumPy array.

So although NVDEC decodes on the GPU, the current integration still transfers frames back to the CPU and then sends them to the GPU again for inference. This is a compatibility bridge, not a fully GPU-resident pipeline. It can be optimized later by using `get_latest()` and preprocessing directly on the GPU.

4.2 Image resizing

```
image1 = cv2.resize(frame1, (3840, 2160))
image2 = cv2.resize(frame2, (3840, 2160))
```

Both images are resized to 3840 x 2160 because downstream processing expects a fixed input resolution.

4.3 CPU-to-GPU transfer

```
cpu_buf1.copy_(torch.from_numpy(image1), non_blocking=True)
cpu_buf2.copy_(torch.from_numpy(image2), non_blocking=True)

gpu_raw1 = cpu_buf1.to("cuda", non_blocking=True)
gpu_raw2 = cpu_buf2.to("cuda", non_blocking=True)
```

- CPU buffers store the images in system memory.
- GPU tensors store the images in GPU memory.
- `non_blocking=True` requests asynchronous transfers where supported.

What is pinned memory?

```
cpu_buf1 = torch.empty((2160, 3840, 3), dtype=torch.uint8, pin_memory=True)
```

Pinned memory is page-locked CPU memory. It helps CUDA transfer data between CPU and GPU more efficiently.

5. How Socket.IO Works with the New UI Page

Socket.IO is not responsible for video decoding or object detection. It sends information between the backend and frontend in real time.

```
PYTHON BACKEND -> Socket.IO -> NETWORK CONNECTION -> FRONTEND -> VIDEO
DISPLAY
```

5.1 Socket.IO server initialization

```
sio = socketio.Server(async_mode="eventlet", cors_allowed_origins="*")
app = socketio.WSGIApp(sio)
```

- **socketio.Server()** creates the Socket.IO server. It handles client connections, disconnections, incoming and outgoing messages, and namespace communication.
- **async_mode="eventlet"** specifies the asynchronous execution model. Eventlet lets the application handle multiple cooperative tasks.
- **cors_allowed_origins="*"**: CORS (Cross-Origin Resource Sharing) controls which website origins can connect to the backend. `"*"` allows any origin. This is convenient in development, but production deployments should restrict allowed origins.

- **socketio.WSGIApp(sio)** wraps the Socket.IO server inside a WSGI-compatible application, which is what the Eventlet web server serves. Without the correct wrapper, the frontend cannot reach the Socket.IO server through its expected HTTP and WebSocket endpoints.

5.2 How the backend starts listening

```
eventlet.wsgi.server(eventlet.listen(("0.0.0.0", 12010)), app)
```

Term	Definition	What it does
eventlet.listen()	Creates a listening network socket	Opens the server port
0.0.0.0	Bind address	Allows connections through available network interfaces
12010	Port number	Identifies the backend service
eventlet.wsgi.server()	WSGI server	Handles incoming HTTP and Socket.IO requests
app	WSGI application	Routes requests to Socket.IO

Why 0.0.0.0? If the backend machine has IP 192.168.100.20, binding to `127.0.0.1` accepts only local connections, while `0.0.0.0` allows other machines to connect via the network IP, provided the firewall and network permit it.

Why port 12010? The frontend needs to know where the backend is listening, for example:

```
const socket = io("http://192.168.100.20:12010/front");
```

The IP above is an example. Replace it with your actual backend machine IP.

6. Understanding Namespaces: /front and /rear

A namespace is a logical communication channel inside one Socket.IO connection. The code defines:

```
@sio.event(namespace="/front")
def connect(sid, environ):
    print("Front namespace connected:", sid)

@sio.event(namespace="/rear")
def connect(sid, environ):
    print("Rear namespace connected:", sid)
```

This gives two separate communication channels:

```
      SOCKET.IO SERVER
            |
   +--------+--------+
   |                 |
 /front            /rear
   |                 |
 FRONT UI          REAR UI
   |                 |
 CCD / IR          CCD / IR
```

What is sid? Session ID, a unique identifier assigned to a connected Socket.IO client within a namespace (for example, Front connected: ABC123, Rear connected: XYZ456). It identifies individual connections.

What is environ? Information about the underlying WSGI connection and HTTP request environment, including request headers and connection-related details.

7. How the Frontend Receives Video Frames

```
sio.emit("frame", payload_front, namespace="/front")
sio.emit("frame", payload_back, namespace="/rear")
```

`emit()` sends an event from the backend to connected clients.

Parameter	Meaning
"frame"	Event name
payload_front	Data being transmitted
namespace="/front"	Destination namespace

For the rear camera the namespace changes to `/rear`.

What is an event? A named message exchanged between backend and frontend. Examples: `frame`, `set_detection`, `set_classification`, `set_camera_mode`, `set_event`. Backend and frontend must agree on event names and payload formats.

8. How an Image Reaches the Frontend

```
def encode_frame(frame):
    buf = jpeg_turbo.encode(frame, quality=70)
    return base64.b64encode(buf).decode()
```

Step 1: JPEG encoding. The original image is converted into compressed JPEG data. A raw 3840 x 2160 image needs a significant amount of memory, and JPEG reduces the data transmitted.

Step 2: Base64 encoding. `base64.b64encode(buf).decode()` converts binary data into a text representation so the payload can contain the image as a string.

Step 3: Payload creation.

```
payload_front = {
    "message_id": frame_id1,
    "frame_id": frame_id1,
    "image": frame_in_bytes_front,
    "targets": []
}
```

Field	Purpose
message_id	Identifies the message
frame_id	Identifies the processed frame
image	Encoded JPEG image
targets	Detection results

Step 4: Send to frontend with `sio.emit("frame", payload_front, namespace="/front")`.

Step 5: Frontend receives the event. The JavaScript might look like:

```
const socket = io("http://192.168.100.20:12010/front");

socket.on("connect", () => {
    console.log("Front connected");
});

socket.on("frame", (data) => {
    console.log("Received frame:", data.frame_id);
    const image = document.getElementById("video");
    image.src = "" + data.image;
});
```

Important: This JavaScript is an illustrative example. Your actual frontend must retain its existing payload handling and display logic.

9. How the Frontend Controls Detection and Classification

The backend also receives events from the UI.

```
@sio.on("set_detection", namespace="/front")
def set_detection_front(sid, data):
    DETECTION_STATE["/front"] = bool(data.get("enabled", False))
```

When the user clicks Detection ON, the frontend sends:

```
socket.emit("set_detection", { enabled: true });
```

The backend sets `DETECTION_STATE["/front"] = True`, and the inference pipeline can use this variable to decide whether detection should run. Classification works the same way:

```
@sio.on("set_classification", namespace="/front")
def set_classification_front(sid, data):
    CLASSIFICATION_STATE["/front"] = bool(data.get("enabled", False))
```

Complete communication flow

```
FRONTEND: user clicks ON
   -> set_detection
   -> SOCKET.IO
   -> BACKEND: DETECTION_STATE = True
   -> Run Detection -> Generate Results
   -> emit(frame)
   -> FRONTEND: display detection
```

One detail worth checking: the event handlers update the state variables, but the inference loop must actually read and apply those variables. Merely defining the handlers does not automatically activate detection.

10. New Technical Terms – Complete Glossary

Term	Definition	Where used
NVDEC	NVIDIA hardware video decoder	Video decoding
PyAV	Python FFmpeg interface	Packet reading
Demuxer	Separates multimedia data into packets	Stream opening
Packet	Compressed video data	Decoder input
PacketData	NVIDIA packet representation	make_packet()
CUDA	NVIDIA GPU computing platform	GPU processing
CUDA Stream	Ordered sequence of GPU operations	Preprocessing/inference
CUDA Event	GPU execution synchronization marker	sync_current_stream()
DLPack	Framework-independent tensor sharing protocol	torch.from_dlpack()
Tensor	Multidimensional data structure	PyTorch
Clone	Creates a separate tensor copy	NVDEC frame retrieval
Thread	Independent execution path	Camera readers
Daemon Thread	Background thread that does not prevent process exit	Reader initialization
Latest Frame Buffer	Stores the newest decoded image	RTSPReader
NumPy	Python numerical computing library	Image processing
BGR	Blue-Green-Red image channel order	OpenCV
RGB	Red-Green-Blue image channel order	NVDEC output
Pinned Memory	Page-locked CPU memory	cpu_buf1, cpu_buf2
H2D	Host-to-Device memory transfer	CPU to GPU
D2H	Device-to-Host memory transfer	GPU to CPU

Term	Definition	Where used
Socket.IO	Real-time bidirectional communication library	Backend/UI
Event	Named communication message	frame, set_detection
Namespace	Logical communication channel	/front, /rear
SID	Session identifier	Connection handlers
WSGI	Python web-server interface	Socket.IO application
Eventlet	Cooperative asynchronous networking library	Server
CORS	Cross-Origin Resource Sharing	Server configuration
Emit	Send an event	sio.emit()
Listener	Receives an event	socket.on()
JPEG	Compressed image format	Frame encoding
Base64	Binary-to-text encoding	Image transmission
Payload	Data sent with an event	payload_front
RTSP	Real-Time Streaming Protocol	Camera connection
ROI	Region of Interest	Detection processing
RF-DETR	Real-time transformer-based object detection architecture	Detection
CCD	Charge-Coupled Device camera	Day camera
IR	Infrared camera	Night/thermal imaging

11. What Changed Between the Original and Integrated Files?

Component	Original hw_v3.py	Integrated file
Video decoding	Existing PyAV-based reader	Intended NVDEC-based reader
GPU decoding	Not using the new NVDEC pipeline	NVDEC
Frame storage	Reader's latest-frame mechanism	Intended GPU tensor buffer

Component	Original hw_v3.py	Integrated file
Frame retrieval	Existing reader API	get_latest_bgr()
Detection	RF-DETR	Preserved
Classification	CCD/IR classifiers	Preserved
Fire/smoke	Existing analytics	Preserved
Human conflict	Existing analytics	Preserved
Socket.IO initialization	Existing	Preserved
Socket namespaces	/front, /rear	Preserved
Event names	Existing	Preserved
Server port	12010	12010
Frontend payload	Existing JSON structure	Intended to be preserved

Why did the new UI page start receiving frames? The Socket.IO implementation was already present in the original hw_v3.py. The integrated file retains the same communication structure:

1. Server initialization: `sio = socketio.Server(...)` and `app = socketio.WSGIApp(sio)`
2. Server startup: `eventlet.listen(("0.0.0.0", 12010))`
3. Front namespace: `namespace="/front"`
4. Rear namespace: `namespace="/rear"`
5. Frame transmission: `sio.emit("frame", payload_front, namespace="/front")` and `sio.emit("frame", payload_back, namespace="/rear")`

The Socket.IO protocol itself was not newly introduced by NVDEC. The intention was to replace the video decoding mechanism while maintaining frontend compatibility.

12. Important Issues Found in the Integrated File

The actual generated hw_v3_nvdec_socketio.py was checked against both reference files. These issues need attention.

Issue 1: RTSPReader class is commented out

The NVDEC reader implementation appears inside a large commented section (for example `# class RTSPReader:`), but at the bottom the program executes:

```
cam_CCD_FRONT = RTSPReader(...)
```

This will cause `NameError: name 'RTSPReader' is not defined`. The complete NVDEC class from nvdec (2).py needs to be restored as executable code.

Issue 2: get_latest_bgr() compatibility

The inference thread calls `cam_CCD_FRONT.get_latest_bgr()`. This method exists in the NVDEC reference file, but is not available while the reader class remains commented out.

Issue 3: Unnecessary GPU-to-CPU-to-GPU transfer

Currently:

```
NVDEC -> GPU Frame -> CPU NumPy Frame -> GPU Tensor -> Inference
```

This adds extra memory transfers. The ideal architecture is:

```
NVDEC -> GPU Frame -> GPU Preprocessing -> GPU Inference
```

This should be the next performance improvement.

Issue 4: Camera fallback logic

The code currently contains:

```
if frame1 is None or frame2 is None:
    continue
```

Later, it tries to replace an unavailable camera with a black frame. However, the `continue` prevents that fallback logic from executing when either frame is missing. The order should be corrected if single-camera operation is required.

Issue 5: Two different server ports

```
def start_server():
    sio.start_background_task(emit_worker)
    eventlet.wsgi.server(eventlet.listen(('0.0.0.0', 5000)), app)
```

The main block starts the server on port 12010, so there are two server-startup paths. Only one should be used consistently. The current main block uses port 12010.

13. Final Understanding: How Everything Works Together

There are three major layers.

Layer 1 – Video acquisition

```
Camera -> RTSP -> PyAV -> NVDEC -> GPU Frame
```

Purpose: get video frames efficiently.

Layer 2 – Video analytics

```
GPU Frame -> Preprocessing -> Detection -> Classification -> Analytics ->
Result Generation
```

Purpose: identify vessels and other relevant events.

Layer 3 – Frontend communication

```
Processed Frame -> JPEG Encoding -> Base64 Encoding -> Socket.IO -> /front
and /rear
```

Purpose: display processed video and detection information on the UI.

Recommended Next Step

Before deploying this version, fix the commented-out NVDEC RTSPReader class and then validate the complete integration. Suggested test sequence:

1. Start all four NVDEC camera readers.
2. Verify that frames are being decoded.
3. Verify that the GPU inference thread receives frames.
4. Start the Socket.IO server on port 12010.
5. Connect the new frontend page to /front and /rear.
6. Verify that JPEG frames arrive continuously.
7. Enable detection and classification from the UI.
8. Verify that the returned payload contains the correct vessel detections and classifications.

One important distinction: the previous generated file is not yet ready for production because the NVDEC reader class is commented out. The Socket.IO architecture is preserved, but the complete integration still needs this correction and runtime testing.